Woche 07 – Übungen

Shor, Ordnungssuche, QFT und Public-Key-Risiko

Softwarestand für Aufgaben 7–14 und 18: Python, NumPy, Matplotlib sowie `qiskit[all]~=2.5.1` oder neuer. Verwendet werden `QuantumCircuit`, `qiskit.quantum_info.Statevector`, `Operator`, `QFTGate`, `PhaseGate`, `UnitaryGate` und die Funktion `phase_estimation`. Die ältere Klasse `PhaseEstimation` wird nicht für neue Lösungen verwendet. Dokumentation: https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector, https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.UnitaryGate, https://quantum.cloud.ibm.com/docs/en/api/qiskit/dev/qiskit.circuit.library.phase_estimation und https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering.

Registerkonvention: Mathematisch wird ein Registerwert als Ganzzahl $x = \sum_j x_j 2^j$ und als Ket $|x_{t-1} \cdots x_0\rangle$ geschrieben. In Qiskit ist q_0 das niederwertigste Bit und steht in ausgegebenen Bitstrings rechts. Ein Statevector-Index x bezeichnet den Rechnerbasiszustand mit ganzzahligem Wert x.

Lernziele

LZ1 Modulare Inversen, Ordnungen und die klassische Faktorisierungsnachverarbeitung berechnen.
LZ2 QFT-Matrizen, Wurzelsummen und Fourieridentitäten herleiten und prüfen.
LZ3 Phasenschätzung und ihre Wahrscheinlichkeitsverteilung berechnen.
LZ4 QFT, Phasenschätzung und modulare Multiplikation reproduzierbar in Python/Qiskit implementieren.
LZ5 Kettenbruch-, Nenner-, Registerordnungs-, Initialisierungs- und Samplingfehler diagnostizieren.
LZ6 Shors Wirkung auf RSA, endliche-Körper-DLP und ECDLP mathematisch analysieren.
LZ7 Algorithmischen Satz, Ressourcenabschätzung, Hardwareevidenz und konkretes Unternehmensrisiko trennen.

Grundlagen und mathematische Sicherheit

Aufgabe 1 – Erweiterter Euklid und modulare Inverse

Schwierigkeit: Basis **Arbeitsform:** Papier
Zeit: 25 Minuten **Lernziel(e):** LZ1

a) Führen Sie den erweiterten euklidischen Algorithmus für 3120 und 17 vollständig aus.

b) Bestimmen Sie ganze Zahlen u, v mit

$$17u + 3120v = 1.$$

c) Bestimmen Sie daraus $17^{-1} \bmod 3120$.

d) Prüfen Sie das Ergebnis durch eine Kongruenz.

e) Beweisen Sie allgemein: a besitzt genau dann ein multiplikatives Inverses modulo m, wenn $\gcd(a, m) = 1$.

f) Erklären Sie in einem Satz, warum diese Rechnung nach erfolgreicher RSA-Faktorisierung benötigt wird.

Hinweis / Selbstkontrolle

Prüfsumme: Das kleinste positive Inverse ist 2753.

Aufgabe 2 – Ordnungen in $\mathbb{Z}_{15}^{\times}$ und $\mathbb{Z}_{21}^{\times}$

Schwierigkeit: Basis **Arbeitsform:** Papier
Zeit: 25 Minuten **Lernziel(e):** LZ1

a) Bestimmen Sie für

$$a \in \{2, 4, 7, 8, 11, 13, 14\}$$

jeweils $\mathrm{ord}_{15}(a)$.

b) Gruppieren Sie die Elemente nach ihrer Ordnung.

c) Bestimmen Sie für $a \in \{2, 4, 8\}$ die Ordnung modulo 21.

d) Prüfen Sie in allen Fällen, dass die Ordnung die Gruppenordnung $\varphi(N)$ teilt.

e) Erklären Sie, warum aus $a^r \equiv 1 \pmod{N}$ nicht folgt, dass jedes Vielfache oder jeder Teiler von r die Ordnung ist.

f) Bestimmen Sie die Perioden der Folgen $x \mapsto 2^x \bmod 15$ und $x \mapsto 4^x \bmod 21$.

Hinweis / Selbstkontrolle

Prüfsummen:

$$\mathrm{ord}_{15}(2) = \mathrm{ord}_{15}(7) = \mathrm{ord}_{15}(8) = \mathrm{ord}_{15}(13) = 4,$$

$$\mathrm{ord}_{15}(4) = \mathrm{ord}_{15}(11) = \mathrm{ord}_{15}(14) = 2, \qquad \mathrm{ord}_{21}(2) = 6, \quad \mathrm{ord}_{21}(4) = 3, \quad \mathrm{ord}_{21}(8) = 2.$$

Aufgabe 3 – Erfolgreiche und erfolglose Shor-Basen

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ1

Für $N = 33$ untersuchen Sie die Basen

$$a \in \{2, 4, 5, 7, 10\}.$$

a) Prüfen Sie jeweils $\gcd(a, 33)$.

b) Bestimmen Sie $r = \mathrm{ord}_{33}(a)$.

c) Klassifizieren Sie den Versuch als erfolgreich, „Ordnung ungerade“ oder „Halbpotenz -1“.

d) Berechnen Sie bei erfolgreichen Basen

$$d_- = \gcd(a^{r/2} - 1, 33), \qquad d_+ = \gcd(a^{r/2} + 1, 33).$$

e) Beweisen Sie, dass bei geradem r und $a^{r/2} \not\equiv \pm 1 \pmod{N}$ beide ggT-Werte nichttrivial sind.

f) Zeigen Sie durch Gegenbeispiel, dass „r gerade“ allein nicht genügt.

g) Untersuchen Sie alle $a \in \mathbb{Z}_9^\times$. Zeigen Sie, dass kein Ordnungssuchversuch die Faktorisierungsbedingung erfüllt, und begründen Sie daraus die klassische Primzahlpotenzprüfung vor dem Quantenanteil.

Hinweis / Selbstkontrolle

$a = 2$ scheitert an der Halbpotenz -1, $a = 4$ an ungerader Ordnung. $a = 5$, 7 und 10 führen zu den Faktoren 3 und 11. Für $N = 9$ treten nur ungerade Ordnungen oder die Halbpotenz -1 auf; zugleich ist $9 = 3^2$ klassisch als perfekte Potenz erkennbar.

Aufgabe 4 – Die Matrix F_4 und ihre Potenzen

Schwierigkeit: Basis **Arbeitsform:** Papier
Zeit: 30 Minuten **Lernziel(e):** LZ2

Sei $\omega_4 = i$ und

$$(F_4)_{y,x} = \frac{1}{2}\omega_4^{xy}, \qquad x, y \in \{0, 1, 2, 3\}.$$

a) Schreiben Sie F_4 vollständig als Matrix.

b) Berechnen Sie $F_4 |0\rangle$, $F_4 |1\rangle$, $F_4 |2\rangle$ und $F_4 |3\rangle$.

c) Zeigen Sie durch Skalarprodukte der Spalten, dass F_4 unitär ist.

d) Zeigen Sie

$$F_4^2 |x\rangle = |-x \bmod 4\rangle .$$

e) Folgern Sie $F_4^4 = I_4$.

f) Bestimmen Sie F_4^{-1} und wenden Sie es auf

$$\frac{1}{2}(1, -i, -1, i)^\mathsf{T}$$

an.

Hinweis / Selbstkontrolle

Für Teil f) ist das Ergebnis ein Rechnerbasiszustand. Achten Sie auf das Vorzeichen im Exponenten der inversen QFT.

Aufgabe 5 – Wurzelsummen und Unitarität von F_Q

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 30 Minuten **Lernziel(e):** LZ2

Für $Q \geq 2$ sei $\omega_Q = \mathrm{e}^{2\pi i/Q}$.

a) Beweisen Sie für $d \in \mathbb{Z}$

$$\sum_{y=0}^{Q-1} \omega_Q^{dy} = \begin{cases} Q, & d \equiv 0 \pmod{Q}, \\ 0, & d \not\equiv 0 \pmod{Q}. \end{cases}$$

b) Leiten Sie daraus $F_Q^\dagger F_Q = I_Q$ her.

c) Zeigen Sie

$$F_Q^{-1}\left(\frac{1}{\sqrt{Q}}\sum_{x=0}^{Q-1}\omega_Q^{mx}\,|x\rangle\right) = |m\rangle\,.$$

d) Beweisen Sie $F_Q^2\,|x\rangle = |-x \bmod Q\rangle$.

e) Erklären Sie, warum eine Messung von $F_Q\,|x\rangle$ in der Rechnerbasis eine Gleichverteilung liefert, obwohl F_Q für Phasenschätzung nützlich ist.

f) Unterscheiden Sie mathematisch „Fouriertransformation der Amplituden“ und „Ausgabe aller Fourierkoeffizienten als klassische Liste“.

Aufgabe 6 – Phasenschätzung: exakter und nicht exakter Fall

Schwierigkeit: Aufbau **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ3

Ein unitärer Operator besitzt einen Eigenzustand $|u\rangle$ mit

$$U\,|u\rangle = \mathrm{e}^{2\pi i\varphi}\,|u\rangle\,.$$

Es werden $t = 3$ Phasenqubits verwendet, also $Q = 8$.

a) Leiten Sie den Zustand unmittelbar vor der inversen QFT her.

b) Bestimmen Sie die Messverteilung für $\varphi = 3/8$.

c) Leiten Sie für allgemeines φ die Amplitude

$$A_y = \frac{1}{8}\sum_{x=0}^{7}\mathrm{e}^{2\pi i x(\varphi - y/8)}$$

her.

d) Berechnen Sie für $\varphi = 1/3$ die Wahrscheinlichkeiten $p(2)$, $p(3)$ und $p(4)$.

e) Bestimmen Sie den wahrscheinlichsten Messwert und die zugehörige Phasenschätzung.

f) Erklären Sie, warum eine höhere Zahl von Phasenqubits zugleich höhere kontrollierte Potenzen von U verlangt.

Hinweis / Selbstkontrolle

Für $\varphi = 1/3$ gilt näherungsweise

$$p(2) = 0.17494, \qquad p(3) = 0.68784, \qquad p(4) = 0.04688.$$

Python und Qiskit

Aufgabe 7 – Modulare Ordnung und Shor-Nachverarbeitung in Python

Schwierigkeit: Basis **Arbeitsform:** Python
Zeit: 25 Minuten **Lernziel(e):** LZ1, LZ4

Implementieren Sie ohne externe Zahlentheoriepakete zwei Funktionen:

`order_mod(a, N)` und `factor_from_order(a, N, r)`.

a) `order_mod` soll bei $\gcd(a, N) \neq 1$ eine klare Ausnahme auslösen und sonst die kleinste positive

Ordnung zurückgeben.

b) Verwenden Sie ausschließlich modulare Potenzierung; große Zwischenwerte sollen vermieden werden.

c) `factor_from_order` soll ungerade r, falsche Ordnungskandidaten und den Fall $a^{r/2} \equiv -1 \pmod{N}$ unterscheiden.

d) Erzeugen Sie eine Tabelle aller $a \in \{2, \ldots, N-2\}$ für $N = 15, 21, 33, 35$.

e) Zählen Sie pro N: unmittelbarer ggT-Treffer, erfolgreicher Ordnungsversuch, ungerade Ordnung und Halbpotenz -1.

f) Schreiben Sie mindestens sechs Assertions für bekannte Fälle aus den Aufgaben 2 und 3.

Hinweis / Selbstkontrolle

Nutzen Sie `pow(a, r, N)` statt `a**r % N`. Die Funktion soll die mathematischen Fehlfälle sichtbar machen und nicht stillschweigend einen beliebigen Teiler zurückgeben.

Aufgabe 8 – Kettenbrüche, Nennerkandidaten und Verifikation

Schwierigkeit: Aufbau **Arbeitsform:** Python
Zeit: 35 Minuten **Lernziel(e):** LZ4, LZ5

Verwenden Sie aus der Standardbibliothek

```
from fractions import Fraction
from math import gcd, lcm
```

Für $N = 21$, $a = 2$, $Q = 512$ seien mögliche Messwerte

$$y \in \{0, 85, 171, 256, 341, 427\}.$$

a) Berechnen Sie mit `Fraction(y, Q).limit_denominator(N)` jeweils einen gekürzten rationalen Kandidaten.

b) Extrahieren Sie dessen Nenner q und prüfen Sie `pow(a, q, N)`.

c) Testen Sie, falls erforderlich, kleine Vielfache $mq < N$ und geben Sie die kleinste bestätigte Ordnung aus.

d) Erklären Sie für jeden Messwert, welcher Eigenphasenindex s plausibel ist.

e) Kombinieren Sie bestätigte Teilerkandidaten aus mehreren Messungen über das kgV und prüfen Sie das Ergebnis erneut.

f) Zeigen Sie an einem künstlichen Messwert, warum `limit_denominator` ohne modulare Verifikation einen falschen Nenner liefern kann.

Hinweis / Selbstkontrolle

Prüfsummen:

$$85/512 \rightsquigarrow 1/6, \quad 171/512 \rightsquigarrow 1/3, \quad 256/512 = 1/2, \quad 427/512 \rightsquigarrow 5/6.$$

Der Messwert 0 enthält keine Ordnungsinformation.

Aufgabe 9 – QFTGate gegen die analytische QFT prüfen

Schwierigkeit: Basis **Arbeitsform:** Qiskit
Zeit: 30 Minuten **Lernziel(e):** LZ2, LZ4

Verwenden Sie

```python
import numpy as np
from qiskit.circuit.library import QFTGate
from qiskit.quantum_info import Operator
```

Für $t = 2, 3, 4$:

a) Erzeugen Sie `QFTGate(t)` und lesen Sie seine Matrix mit `Operator(...).data` aus.

b) Konstruieren Sie unabhängig die Referenzmatrix

$$(F_Q)_{y,x} = Q^{-1/2} \mathrm{e}^{2\pi i x y / Q}.$$

c) Prüfen Sie numerisch die maximale Elementabweichung.

d) Prüfen Sie $F_Q^\dagger F_Q = I_Q$ und $F_Q^4 = I_Q$.

e) Wenden Sie `QFTGate(t).inverse()` auf das lineare Phasenmuster zu $m = 3$ an.

f) Formulieren Sie Tests, die einen vertauschten Exponentenvorzeichen-Fehler sicher erkennen.

Hinweis / Selbstkontrolle

Bei doppelter Genauigkeit sollten die Matrixfehler in der Größenordnung 10^{-14} bis 10^{-12} liegen. Vergleichen Sie Matrizen, nicht nur Messwahrscheinlichkeiten: Vorzeichenfehler können dieselbe Gleichverteilung erzeugen.

Aufgabe 10 – Exakte Phasenschätzung mit der aktuellen API

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 35 Minuten **Lernziel(e):** LZ3, LZ4

Nutzen Sie die aktuelle Funktionsschnittstelle:

```python
import numpy as np
from qiskit import QuantumCircuit
from qiskit.circuit.library import PhaseGate, phase_estimation
from qiskit.quantum_info import Statevector

phi = 3 / 8
t = 4
unitary = PhaseGate(2 * np.pi * phi)
qpe = phase_estimation(t, unitary)

qc = QuantumCircuit(t + 1)
qc.x(t)  # Systemqubit im Eigenzustand |1>
qc.compose(qpe, qubits=range(t + 1), inplace=True)
sv = Statevector.from_instruction(qc)
probs = sv.probabilities_dict(qargs=list(range(t)))
print(probs)
```

a) Begründen Sie, warum $|1\rangle$ Eigenzustand des `PhaseGate` ist.

b) Sagen Sie das Messergebnis als Ganzzahl und Bitstring vorher.

c) Führen Sie den Code aus und prüfen Sie die Vorhersage.

d) Wiederholen Sie für $\varphi \in \{1/16, 5/16, 7/8\}$.

e) Ersetzen Sie die Vorbereitung $|1\rangle$ durch $|0\rangle$ und erklären Sie das Ergebnis.

f) Schreiben Sie Assertions, die Normierung, eindeutiges Peak-Ergebnis und korrekte Ganzzahldekodierung prüfen.

Hinweis / Selbstkontrolle

Für $\varphi = 3/8$ und $Q = 16$ ist $Q\varphi = 6$. Der erwartete angezeigte Bitstring ist daher `0110`.

Aufgabe 11 – Modulare Multiplikation als Permutationsmatrix

Schwierigkeit: Aufbau **Arbeitsform:** Hybrid
Zeit: 40 Minuten **Lernziel(e):** LZ4, LZ5

Für $N = 15$, $a = 2$ und $n = 4$ soll der Operator

$$U_a |y\rangle = \begin{cases} |2y \bmod 15\rangle, & 0 \le y < 15, \\ |15\rangle, & y = 15 \end{cases}$$

implementiert werden.

a) Erstellen Sie zunächst auf Papier die Abbildungstabelle für $y = 0, \ldots, 15$.

b) Erzeugen Sie eine 16×16-Matrix U mit genau einem Eintrag 1 pro Spalte: $U_{z,y} = 1$, falls $|y\rangle \mapsto |z\rangle$.

c) Prüfen Sie algorithmisch, dass jede Zeile und jede Spalte genau eine Eins enthält.

d) Erzeugen Sie

```
from qiskit.circuit.library import UnitaryGate
from qiskit.quantum_info import Operator, Statevector

gate = UnitaryGate(U, label="x2 mod 15")

```

und prüfen Sie Unitarität.

e) Testen Sie die Wirkung auf alle 16 Rechnerbasiszustände mit `Statevector.from_int(...).evolve(gate)`.

f) Bestimmen Sie die Zykelzerlegung der Permutation und lesen Sie daraus die Ordnung der Bahn von $|1\rangle$ ab.

Hinweis / Selbstkontrolle

Die relevante Bahn ist

$$1 \to 2 \to 4 \to 8 \to 1.$$

Die Zustände 0 und 15 sind Fixpunkte. Verwechseln Sie Matrixzeile und Matrixspalte nicht.

Aufgabe 12 – Vollständige Toy-Ordnungssuche für $N = 15$

Schwierigkeit: Transfer **Arbeitsform:** Qiskit
Zeit: 50 Minuten **Lernziel(e):** LZ4, LZ5

Verwenden Sie den `UnitaryGate` aus Aufgabe 11 und

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import phase_estimation
from qiskit.quantum_info import Statevector

t = 4
n = 4
qpe = phase_estimation(t, gate)
qc = QuantumCircuit(t + n)
qc.x(t)  # Arbeitsregister q[t],...,q[t+n-1] auf Integer 1
qc.compose(qpe, qubits=range(t + n), inplace=True)
sv = Statevector.from_instruction(qc)
```

```
phase_probs = sv.probabilities_dict(qargs=list(range(t)))
```

a) Begründen Sie genau, warum `qc.x(t)` den Arbeitsregisterwert 1 vorbereitet.

b) Sagen Sie die vier Peaks und ihre exakten Wahrscheinlichkeiten vorher.

c) Prüfen Sie die Vorhersage mit `phase_probs`.

d) Ziehen Sie mit festem Seed 4096 Stichproben aus dem Statevector und vergleichen Sie Häufigkeiten mit den exakten Wahrscheinlichkeiten.

e) Implementieren Sie die klassische Nachverarbeitung für jeden beobachteten Peak.

f) Dokumentieren Sie getrennt die Fälle $y = 0$, $y = 4$, $y = 8$ und $y = 12$.

g) Schreiben Sie einen reproduzierbaren Test, der aus mindestens einer geeigneten Stichprobe die Faktoren 3 und 5 gewinnt.

Hinweis / Selbstkontrolle

Im idealen Statevector liegen die Peaks bei

$$y \in \{0, 4, 8, 12\}$$

mit jeweils 1/4. Der Nenner aus $y = 8$ ist zunächst 2 und muss verifiziert beziehungsweise erweitert werden.

Analyse, Debugging und Interpretation

Aufgabe 13 – QPE-Debugging: API, Eigenzustand und Bitordnung

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 35 Minuten **Lernziel(e):** LZ5

Der folgende Code soll $\varphi = 3/8$ schätzen, liefert aber ein irreführendes Resultat:

```
import numpy as np
from qiskit import QuantumCircuit
from qiskit.circuit.library import PhaseEstimation, PhaseGate
from qiskit.quantum_info import Statevector

phi = 3 / 8
t = 4
unitary = PhaseGate(2 * np.pi * phi)
qpe = PhaseEstimation(t, unitary)

qc = QuantumCircuit(t + 1)
qc.compose(qpe, qubits=range(t + 1), inplace=True)
sv = Statevector.from_instruction(qc)
probs = sv.probabilities_dict(qargs=list(range(t)))
best = max(probs, key=probs.get)
estimate = int(best[::-1], 2) / 2**t
print(best, estimate)
```

a) Identifizieren Sie mindestens drei voneinander unabhängige Probleme.

b) Erklären Sie für jedes Problem die mathematische Wirkung auf den Zustand oder die Interpretation.

c) Korrigieren Sie den Code unter Verwendung der Funktion `phase_estimation`.

d) Ergänzen Sie Assertions für $\varphi = 3/8$ und $\varphi = 5/16$.

e) Erzeugen Sie absichtlich einen falschen QFT-Vorzeichenfall und zeigen Sie, welcher Peak dann entsteht.

f) Formulieren Sie einen kurzen Testvertrag für QPE-Implementierungen.

Hinweis / Selbstkontrolle

Zu prüfen sind mindestens: veraltete Klasse, fehlende Vorbereitung des System-Eigenzustands und unnötige Umkehr des ausgegebenen Bitstrings.

Aufgabe 14 – Nicht exakte Phase: Statevector, Shots und Schätzfehler

Schwierigkeit: Aufbau **Arbeitsform:** Qiskit
Zeit: 40 Minuten **Lernziel(e):** LZ3, LZ5

Schätzen Sie

$$\varphi = \frac{1}{3}$$

mit $t = 5$ Phasenqubits.

a) Erzeugen Sie den QPE-Schaltkreis für `PhaseGate(2*pi/3)` und Systemzustand $|1\rangle$.

b) Berechnen Sie die exakte marginale Verteilung des Phasenregisters mit `Statevector.probabilities_dict`.

c) Bestimmen Sie die fünf größten Peaks und vergleichen Sie mit der Formel aus Vorlesung A.

d) Ziehen Sie für Shots $S \in \{128, 512, 2048, 8192\}$ jeweils 50 unabhängige Stichprobenläufe.

e) Untersuchen Sie Mittelwert und Standardabweichung der geschätzten Häufigkeit des Hauptpeaks.

f) Vergleichen Sie den beobachteten Fehler mit der binomialen Näherung

$$\sigma_{\widehat{p}} \approx \sqrt{\frac{p(1-p)}{S}}.$$

g) Erklären Sie, warum endliche Shots kein Hardware-Rauschen sind.

Hinweis / Selbstkontrolle

Für $Q = 32$ ist der Hauptpeak $y = 11$ mit ungefähr 0.68416. Der zweitgrößte Peak liegt bei $y = 10$ mit ungefähr 0.17122.

Aufgabe 15 – QFT-Gatterzahl und Ressourcenebenen

Schwierigkeit: Aufbau **Arbeitsform:** Hybrid
Zeit: 30 Minuten **Lernziel(e):** LZ5, LZ7

Eine exakte QFT auf t Qubits verwendet in der Standardzerlegung

$$t \text{ Hadamard-Gatter}, \qquad \frac{t(t-1)}{2} \text{ kontrollierte Phasengatter}, \qquad \left\lfloor \frac{t}{2} \right\rfloor \text{ SWAPs}$$

oder eine äquivalente klassische Bitumkehr am Ausgang.

a) Berechnen Sie diese Zahlen für $t = 8, 32, 256, 4096$.

b) Bestimmen Sie den Gesamtwert für $t = 4096$ mit und ohne physische SWAPs.

c) Erklären Sie, warum diese Zahl nicht die Gatterzahl einer vollständigen RSA-2048-Faktorisierung ist.

d) Ordnen Sie folgende Größen einer Ebene zu: abstrakte QFT-Gatter, reversible modulare Multiplikation, logische T-/Toffoli-Gatter, Code-Distanz, physische Qubits, Wandzeit.

e) Diskutieren Sie, welche kleinen Rotationen eine approximative QFT weglassen könnte und welche Fehlergröße kontrolliert werden muss.

f) Formulieren Sie drei Gründe, warum eine asymptotische $O(n^3)$-Aussage keine belastbare Hardwarelaufzeit liefert.

Hinweis / Selbstkontrolle

Für $t = 4096$ entstehen 8,386,560 kontrollierte Phasenoperationen und 2048 SWAPs. Modulare Exponentiation und Fehlerkorrektur sind darin nicht enthalten.

Aufgabe 16 – RSA-Schlüsselrekonstruktion nach der Faktorisierung

Schwierigkeit: Aufbau **Arbeitsform:** Hybrid
Zeit: 30 Minuten **Lernziel(e):** LZ1, LZ6

Gegeben sei der öffentliche RSA-Schlüssel

$$N = 3233, \qquad e = 17.$$

Ein Ordnungssuchlauf habe die Faktoren

$$p = 61, \qquad q = 53$$

geliefert.

a) Berechnen Sie $\varphi(N)$.

b) Bestimmen Sie den privaten Exponenten d mit erweitertem Euklid.

c) Prüfen Sie $ed \equiv 1 \pmod{\varphi(N)}$.

d) Verschlüsseln Sie $m = 123$ als $c = m^e \bmod N$ und entschlüsseln Sie mit d.

e) Implementieren Sie die Rechnung mit `pow(base, exponent, modulus)`.

f) Erklären Sie, warum dieselbe Faktorisierung sowohl RSA-Entschlüsselung als auch RSA-Signaturfälschung ermöglicht.

Hinweis / Selbstkontrolle

Prüfsummen:

$$\varphi(N) = 3120, \qquad d = 2753, \qquad c = 855.$$

Transfer, Kryptografie und Evidenz

Aufgabe 17 – Diskreter Logarithmus als zweidimensionale Periode

Schwierigkeit: Transfer **Arbeitsform:** Papier
Zeit: 35 Minuten **Lernziel(e):** LZ6

In $\mathbb{Z}_{23}^{\times}$ sei

$$g = 5, \qquad h = 8 = g^x \pmod{23}.$$

a) Bestimmen Sie klassisch durch eine Potenztabelle den diskreten Logarithmus x.

b) Definieren Sie

$$f(u, v) = g^u h^{-v} \pmod{23}.$$

und zeigen Sie $f(u + x, v + 1) = f(u, v)$.

c) Bestimmen Sie alle Paare $(\Delta u, \Delta v)$ mit

$$\Delta u - x\Delta v \equiv 0 \pmod{22}.$$

d) Beschreiben Sie, welche lineare Information eine zweidimensionale Fourierabtastung über die verborgene Untergruppe liefern muss.

e) Ordnen Sie das Problem Diffie–Hellman und DSA zu.

f) Vergleichen Sie den idealen Shor-Aufwand polynomial in $\log 23$ mit einer generischen klassischen Suche; erklären Sie, warum die Spielzeuggröße keine praktische Aussage erlaubt.

Hinweis / Selbstkontrolle

Der diskrete Logarithmus ist $x = 6$, denn $5^6 \equiv 8 \pmod{23}$.

Aufgabe 18 – Elliptischer Diskretlogarithmus auf einer Spielzeugkurve

Schwierigkeit: Transfer **Arbeitsform:** Python
Zeit: 45 Minuten **Lernziel(e):** LZ6

Betrachten Sie über $\mathbb{F}_{17}$ die Kurve

$$E: \quad y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

und den Punkt

$$P = (5, 1).$$

a) Prüfen Sie, dass P auf E liegt.

b) Implementieren Sie Punktaddition einschließlich neutralem Element, Verdopplung und vertikaler Tangente.

c) Erzeugen Sie die Tabelle kP für $k = 1, \ldots, 19$.

d) Bestimmen Sie die Ordnung von P.

e) Für $Q = (0, 6)$ bestimmen Sie x mit $Q = xP$.

f) Formulieren Sie die verborgene Periodenfunktion

$$f(u, v) = uP - vQ$$

und beweisen Sie $f(u + x, v + 1) = f(u, v)$.

g) Erklären Sie, welche Teile Ihrer Python-Rechnung klassisches Brute Force sind und welcher Teil in Shors ECDLP-Verfahren quantummechanisch ersetzt würde.

Hinweis / Selbstkontrolle

Prüfsummen:

$$19P = \mathcal{O}, \qquad 7P = (0, 6).$$

Testen Sie nach jeder Addition erneut die Kurvengleichung.

Aufgabe 19 – Risikobasierte Public-Key-Systemlandkarte

Schwierigkeit: Transfer **Arbeitsform:** Hybrid
Zeit: 45 Minuten **Lernziel(e):** LZ6, LZ7

Analysieren Sie die folgenden Systeme:

1) TLS 1.3 mit ECDHE und ECDSA-Zertifikat; Transkripte werden zehn Jahre archiviert.

2) VPN mit statischem RSA-Schlüsseltransport; Nutzdaten besitzen zwölf Jahre Vertraulichkeitsbedarf.
3) Code Signing mit RSA-PSS; signierte Firmware bleibt fünfzehn Jahre im Feld.
4) Dokumentensignatur mit EdDSA; die Signatur muss langfristig beweiskräftig bleiben.
5) AES-256-Backup, dessen zufälliger Datenverschlüsselungsschlüssel mit RSA-2048 gekapselt wird.
6) Passwort-Hashing mit Salt und speicherharter Funktion ohne Public-Key-Komponente.

Erstellen Sie für jedes System eine Zeile mit:

- kryptografischer Primitive;
- mathematischer Annahme;
- Shor-relevanter Angriffsschritt;
- betroffener Schutzeigenschaft;
- Möglichkeit eines „aufzeichnen heute, entschlüsseln später“-Angriffs;
- notwendiger technischer Inventarnachweis;
- Priorität niedrig / mittel / hoch mit begründeter Unsicherheit.

Schließen Sie mit drei Regeln, die verhindern, dass „Shor bricht RSA/ECC“ zu einer undifferenzierten Systemaussage verkürzt wird.

Hinweis / Selbstkontrolle

Trennen Sie Schlüsselvereinbarung, Verschlüsselung und Signatur. Bei einer Signatur ist „später entschlüsseln“ nicht die passende Schutzeigenschaft; relevant sind Schlüsselerholung, Fälschung und langfristige Vertrauensanker.

Aufgabe 20 – Theorem, Ressourcenabschätzung und Demonstration

Schwierigkeit: Transfer **Arbeitsform:** Literatur
Zeit: 60 Minuten **Lernziel(e):** LZ7

Lesen Sie gezielt:

- Shor 1994 oder Shor 1997 zur polynomialen Faktorisierung und DLP-Lösung;
- NIST IR 8105 zur kryptografischen Einordnung;
- Gidney–Ekerå 2021 und Gidney 2025 zu RSA-2048-Ressourcenabschätzungen.

Erstellen Sie eine Evidenzmatrix mit den Spalten

Behauptung	Evidenztyp	Annahmen
Messung oder Schätzung	übertragbare Aussage	nicht zulässige Folgerung

Bearbeiten Sie mindestens diese Behauptungen:

a) Faktorisierung ist auf einem idealen Quantencomputer polynomial in der Eingabelänge lösbar.
b) RSA-2048 kann unter einem bestimmten Fehlermodell mit einer bestimmten physischen Ressourcenordnung faktorisiert werden.
c) Ein solcher Rechner existiert gegenwärtig als demonstriertes Angriffssystem.
d) Eine kleinere Ressourcenschätzung liefert automatisch ein belastbares Kalenderdatum für einen Angriff.
e) Unternehmen müssen Verwundbarkeit, Datenlebensdauer und Migrationszeit gemeinsam betrachten.

Schließen Sie mit einer maximal 180 Wörter langen technischen Entscheidungsvorlage, in der jede Aussage als *Satz*, *Engineering-Schätzung*, *beobachtete Demonstration* oder *Risikoinferenz* markiert ist.

Hinweis / Selbstkontrolle

Eine Arbeit mit konkreten Qubit- und Laufzeitwerten kann eine sorgfältige Engineering-Schätzung sein, ohne dass die beschriebene Maschine gebaut oder die Faktorisierung experimentell ausgeführt wurde. Prüfen Sie insbesondere Fehlerrate, Zykluszeit, Konnektivität, Reaktionszeit und Fehlerkorrekturannahmen.

Literatur und Vertiefung

- **Shor (1994):** *Algorithms for quantum computation: discrete logarithms and factoring*, DOI 10.1109/SFCS.1994.365700. Für die ursprüngliche algorithmische Idee und die gemeinsame Struktur von Faktorisierung und diskretem Logarithmus.
- **Shor (1997):** *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*, DOI 10.1137/S0097539795293172. Für die ausführlichere mathematische Fassung und Komplexitätsaussagen.
- **Nielsen–Chuang, Kapitel 5:** Für QFT, Phasenschätzung, Ordnungssuche und die Standardanalyse der Erfolgswahrscheinlichkeit.
- **Kitaev (1995):** arXiv:quant-ph/9511026. Für die allgemeine Verbindung zwischen Phasenschätzung und abelschen Stabilisatorproblemen.
- **Proos–Zalka (2003):** arXiv:quant-ph/0301141. Für die Übertragung auf elliptische Kurven und konkrete ECDLP-Ressourcenüberlegungen.
- **Gidney–Ekerå (2021):** DOI 10.22331/q-2021-04-15-433. Für ein explizites fehlertolerantes Ressourcenmodell; Annahmen und Schätzung von experimenteller Demonstration trennen.
- **Gidney (2025):** arXiv:2505.15917. Für eine neuere, veränderte Raum-Zeit-Abwägung bei RSA-2048; als Preprint und Engineering-Schätzung kritisch lesen.
- **NIST IR 8105:** DOI 10.6028/NIST.IR.8105. Für Public-Key-Auswirkungen, Infrastrukturkontext und die Begründung frühzeitiger kryptografischer Planung.
- **IBM Quantum Dokumentation:** https://quantum.cloud.ibm.com/docs/en/api/qiskit/dev/qiskit.circuit.library.phase_estimation, https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.UnitaryGate und https://quantum.cloud.ibm.com/docs/en/guides/bit-ordering. Für die aktuell verwendeten Qiskit-Schnittstellen und Registerkonventionen.